RSE day 2

Welcome back!

Outline

DAY 2 - morning		
Topic	**Time**	**Duration**
Welcome	09:30 – 09:45	0:15
Automation & Introduction to testing	09:45 – 10:15	0:30
Break	10:30 – 11:15	0:15
Testing exercise	11:15 – 11:30	0:45
Break	11:15 – 11:30	0:15
Quick introduction to CI/CD + Activity 2	11:30 – 12:30	1:00
Lunch	12:30 - 13:30	0:30

Automation

Makefiles

- build automation tool
- Language agnostic - > bash commands
- Smart execution of pipelines, recognizes inputs and outputs
- Consists of a set of rules, such like this:

Makefile

```
targets: prerequisites
  command
  command
  command
```


https://makefiletutorial.com/

Makefiles

```
# Makefile

.PHONY: all clean

# Define filenames
RAW_DATA = data/student_habits_performance.csv
CLEANED_DATA = output/clean_data.csv
PLOT_IMAGE = output/plot.png

# Rule to create cleaned data
$(CLEANED_DATA): src/clean_data.py $(RAW_DATA)
    python3 src/clean_data.py $(RAW_DATA) $(CLEANED_DATA)

# Rule to create plot
$(PLOT_IMAGE): src/plot_data.py $(CLEANED_DATA)
    python3 src/plot_data.py $(CLEANED_DATA) $(PLOT_IMAGE)

# "all" target: runs both
all: $(PLOT_IMAGE)
```

PHONY: by default, makefiles check if the target file exists and only execute if not

- Targets under .PHONY are excluded from that rule

Example in solution day 1.2

Good coding practices - automation

Snakemake (python)

- Allows to schedule scientific workflows, data analysis pipelines, especially in bioinformatics and computational science.
- Install using conda

```
rule clean_data:
    input: "raw_data.csv"
    output: "clean_data.csv"
    shell: "python clean_data.py {input} {output}"

rule plot:
    input: "clean_data.csv"
    output: "plot.png"
    shell: "python plot_data.py {input} {output}"
```

Introduction to Testing

It’s normal to make mistakes…

NEWS OF THE WEEK

A Scientist's Nightmare: Software Problem Leads to Five Retractions

GREG MILLER Authors Info & Affiliations

SCIENCE • 22 Dec 2006 • Vol 314, Issue 5807 • pp. 1856-1857 • DOI: 10.1126/science.314.5807.1856

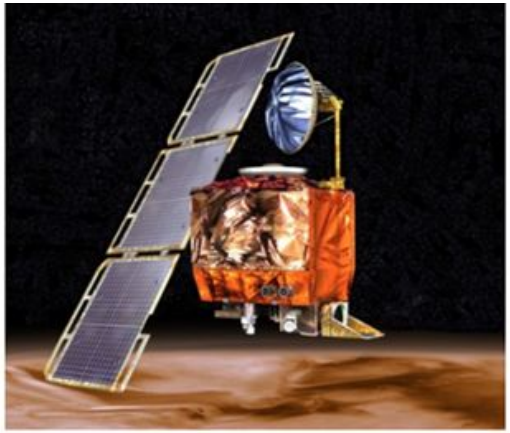
Until recently, Geoffrey Chang's career was on a trajectory most young scientists only dream about. In 1999, at the age of 28, the protein crystallographer landed a faculty position at the prestigious Scripps Research Institute in San Diego, California. The next year, in a ceremony at the White House, Chang received a Presidential Early Career Award for Scientists and Engineers, the country's highest honor for young researchers. His lab generated a stream of high-profile papers detailing the molecular structures of important proteins embedded in cell membranes.

Then the dream turned into a nightmare. In September, Swiss researchers published a

It’s normal to make mistakes…

LISA GROSSMAN 11.10.10 7:00 AM

NOV. 10, 1999: METRIC MATH MISTAKE MUFFED MARS METEOROLOGY MISSION



The **$125 million satellite** was supposed to be the first weather observer on another world. But as it approached the red planet to slip into a stable orbit Sept. 23, the orbiter vanished. Scientists realized quickly it was gone for good. “It was pretty clear that morning, within half-an-hour, that the spacecraft had more or less **hit the top of the atmosphere and burned up**,” recalled NASA engineer Richard Cook, who was project manager for Mars exploration

It’s normal to make mistakes …

Comment | Open access | Published: 23 August 2016

Gene name errors are widespread in the scientific literature

Mark Ziemann, Yotam Eren & Assam El-Osta

Genome Biology **17**, Article number: 177 (2016) | Cite this article

Abstract

The spreadsheet software Microsoft Excel, when used with default settings, is known to convert gene names to dates and floating-point numbers. A programmatic scan of leading genomics journals reveals that approximately one-fifth of papers with supplementary Excel gene lists contain erroneous gene name conversions.

So you better expect them: Tests

What are Software Tests?

- Software tests are functions that certify the functionality of your code.
- They form a separate code base with your code and are run **independently/after/on** your code base

Verification vs. validation

- **Validation**: This is the process of ensuring the input and output parameters of a product. It answers this question: *Are we building the product correctly*?
- **Verification**: This is the process of checking whether the product fulfills the specified requirements. Verifications answer question: *Are we building the right product?*
- Software test aim at verification, not at validation

Assumptions and silent failure

The issue:

- The correct execution of these programmes is based on **assumptions**
  - Example 2: the unit of the measurement
  - Example 3: the correct naming of the columns

THESE ASSUMPTIONS WERE NOT TESTED

WORSE: the program executed and failed silently.

TESTS allow us to check assumptions and to FAIL LOUDLY when the assumptions are not met

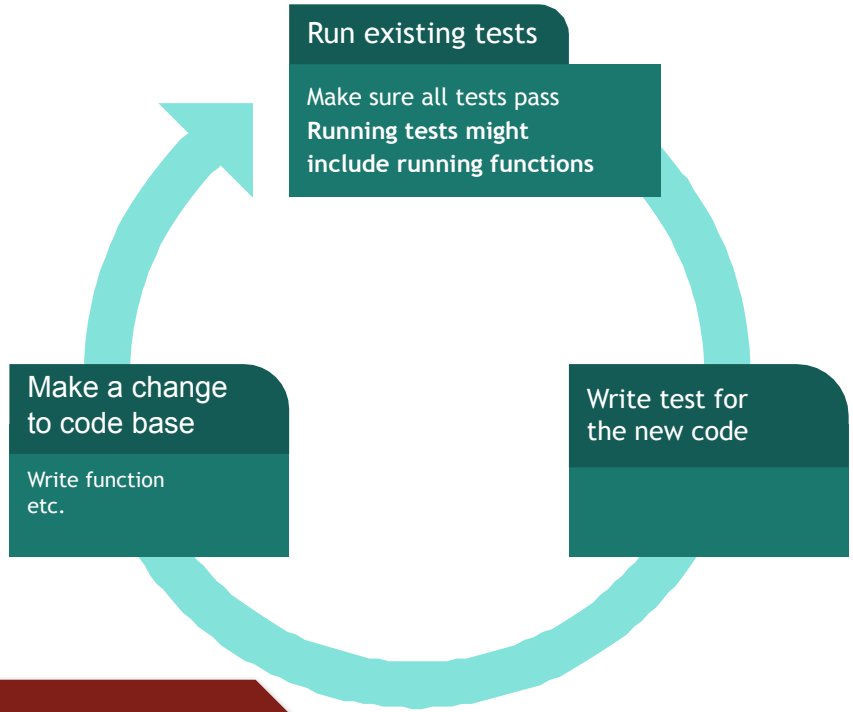
Commonly used validation methods that are not tests

- Print statements
- Plotting images → Visual inspection
- Abort statements

Issues:

- Might get lost in noise of terminal output
- Require expert knowledge to validate them
- All of them are executed at runtime and cannot be run separately from the code

The test cycle



Test cycle

- Tests run you functions independently of what is before and after
- This enforces modularity on your functions & code

Different types of tests - as part of the test cycle

- Unit testing
- Integration testing
- Regression testing
- System testing
- Smoke testing
- Runtime testing

Unit tests

A type of testing in which the **logic of the smallest components or objects of software is tested**. Tests **small parts** of your code. This is predominantly the first type of testing performed in the development life cycle.

Example: You want to make a ball pen

- You test the cap, the ball, the ink etc SEPARATELY

Unit tests

A type of testing in which the **logic of the smallest components or objects of software is tested**. Tests **small parts** of your code. This is predominantly the first type of testing performed in the development life cycle.

Pros:

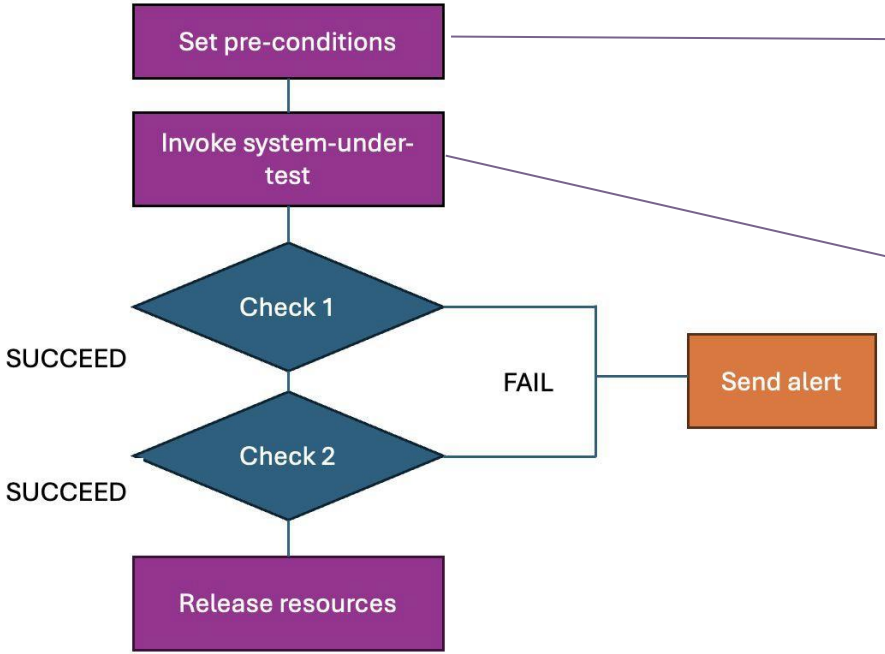
- Small, easy to execute
- Easy to write *if* code base is organized in small functions
- Help to locate bugs precisely, as small part of code base is tests
- Hermetic

Cons:

- Does not test how different parts of the code base interact with each other
- Harder to write if code is complex

Anatomy of a unit test

$$x(t) = x_0 + v_0(t-t_0) + 1/2a(t-t_0)^2$$



$x_0 = 1.0$
$v_0 = 2.0$
$t = 0.5$
$t_0 = 1$

x_t = compute_distance (x_0, v_0, t, t_0)

call assert_equal(x_1, 0.5)

Unit tests

Unit tests - practically

What are attributes of good unit tests

- Silent in the case of success
- Automated and repeatable
- Independent (no side effects)
- Transparent (obvious, but not tautological)
- Narrow/precise
- Orthogonal (one bug => one failing test)
- small/frugal (memory, resources)

Should cover the entire code base -> test coverage

Integration tests

A testing technique in which the communication logic of the individual software components or services are combined and tested as a group.

While unit tests focus on the functionality of individual parts, the focus of integration tests should be on the correct **interaction** between different parts.

Important when different people work on different parts of a system.

Pros:

- Tests the interaction between parts, not only parts themselves

Cons:

- Give only broad idea of where the failure happened or what caused it

Integration tests

A testing technique in which the communication logic of the individual software components or services are combined and tested as a group.

While unit tests focus on the functionality of individual parts, the focus of integration tests should be on the correct interaction between different parts.

Important when different people work on different parts of a system.

Example: *You want to make a ball pen*

- You write with the ball pen

Types of integration test approaches

- **Big Bang**: the integration of all units of a system are tested at the same time
  - Common when testers get a full new code base
- **Top down**: the integration of greater top units are tested first, then the integration of smaller bottom units
- **Bottom up**: the integration of smaller units is tested first, then the integration of higher level units/parts

- test stubs might (mocks, fixtures) need to be used, as the focus is on integration (aka the test should fail due to the interaction of different parts, not because of a failure of a lower order test)

Regression tests

Style of testing that focuses on retesting after changes are made or a bug has been found. The results of tests after the changes are compared to the results before, and errors are raised if these are different.

- Very obviously important when you work in a team
- Written when code is functioning: record the output and save it
    - This output is then compared against the output of running the code later, after changes have been made

Example: *You want to make a pen:*

- You change the ink. So you write before and after to see whether the pen still works as before

Regression tests

- Bug regression: We retest a specific bug that has been allegedly fixed.
- Old fix regression testing: We retest several old bugs that were fixed, to see if they are back. (This is the classical notion of regression: the program has regressed to a bad state.)
- General functional regression: We retest the project broadly, including areas that worked before, to see whether more recent changes have destabilized working code

Pros:

- Good sanity check when working with more than one person

Cons:

- Regression tests show that the code produces correct results, not necessarily that the code is working correctly
- You need to update the reference examples when the code changes fundamentally

https://book.the-turing-way.org/reproducible-research/testing/testing-acceptance-regression

System tests (end-to-end testing)

Run a program from end to end.

Can be automated, for example: run every night

Pros:

- Easy to implement
- Works well even with complex code

Cons:

- Can be resource-consuming (time, memory) etc to run

Smoke tests

A testing technique that is primarily used to check the core features of a product when there is a change introduced or before releasing it to a wider audience.

Run at the beginning of a test cycle

Checks for the bare minimum and big red flags

- Example: Does the code compile?

Pros:

- Easy to implement

Cons:

- Not specific
- Does not check whether results are correct

Runtime testing

Tests that are run as part of the program itself

If statements that have an abort part

```
population = population + people_born - people_died

// test that the population is positive
if (population < 0):
  error( 'The number of people can never be negative' )
```

Runtime testing

internal checks within functions that verify that their inputs and outputs are valid

```
function add_arrays( array1, array2 ):

// test that the arrays have the same size
if (array1.size() != array2.size()):
  error( 'The arrays have different sizes!' )

output = array1 + array2

if (output.size() != array1.size()):
  error( 'The output array has the wrong size!'' )

return output
```

Runtime testing

internal checks within functions that verify that their inputs and outputs are valid

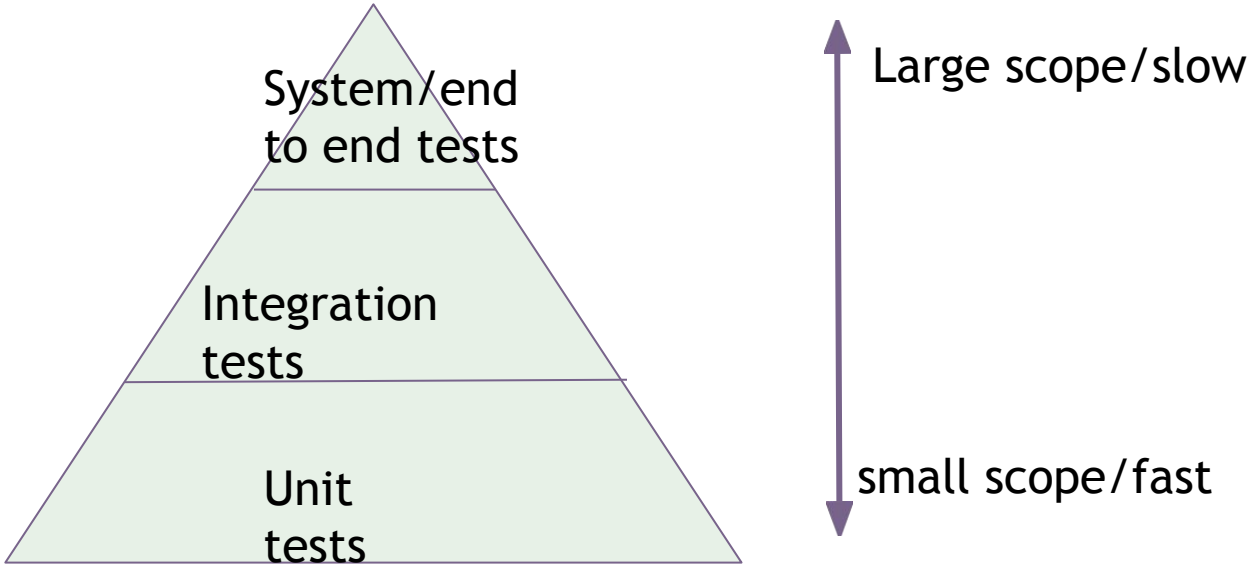
Pros:

- Run within the program, so can catch problems caused by logic errors or edge cases.
- Makes it easier to find the cause of the bug by catching problems early.
- Catching problems early also helps prevent them escalating into catastrophic failures. It minimises the blast radius.

Cons

- Tests can slow down the program.
- What is the right thing to do if an error is detected? How should this error be reported? Exceptions are a recommended route to go with this.
- Program needs to be run

Hierarchy of tests



Distinction by scope and proposed fractions are based on Wacker 2015

Testing Frameworks

	Single case	Mocking	Property based testing	Mutation testing
C++	Catch2 & GoogleTest	GoogleTest	rapidcheck	
Rust	Cargo tests (builtin)	Mockall	Quickcheck & proptest	Cargo-mutants
Python	Pytest & Unittest	Mock	Hypothesis	Mutatest
Java	JUnit	Mockito	jqwik & junit-quickcheck	Pitest
Julia	Unit Testing	Mocking		

Tests can also be run via language agnostic frameworks, such as make and snakemake

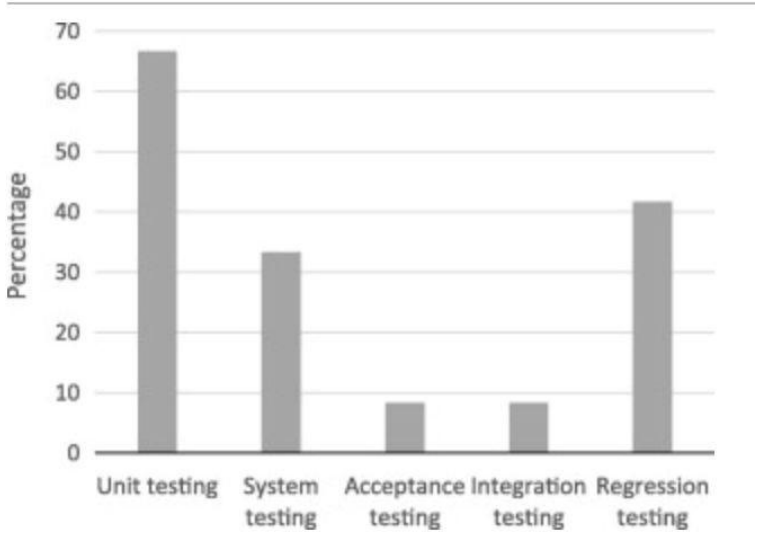
Test driven development

Development paradigm that start with writing the tests

1. Assumption: what is the result/output that my code should deliver?
2. Write the test.
3. Write the code.
4. Run the test.
5. Proceed with 1.

Aims for FULL test coverage

Software tests used in scientific research



Kanewala, U. & Bieman, J. M. Testing scientific software: A systematic literature review. *Inf. Softw. Technol.* **56**, 1219–1232 (2014).

Setting up your test environment

- Tests live in a separate folder to your code base
- They are run independently from your code base

```
your_project/
├── your_package/
│   └── my_module.py
├── tests/
│   └── test_my_module.py
├── requirements.txt
└── pytest.ini  # Optional config
```

Assert - examples

- Assert is part of the standard library (python, matlab)
- Most languages have the equivalent of a assert statement
- Assert statements can be part of a test function, but can also be integrated into the code

```
assert x > 0, "x must be positive"
assert condition, message
assert x > 0
```

- The message is optional and is thrown in case of error
- asserts pass SILENTLY and fail LOUDLY

Testing example

https://jupyter.jsc.fz-juelich.de/user/fritjoflammers_at_gmail.com/fdd8cbfc59244311b6ed2d9847d55ced/lab/workspaces/fdd8cbfc59244311b6ed2d9847d55ced/tree/home/jovyan/RSE_Juelich/day2.1/examples/examples_testing.ipynb

Test fixtures and mocks

Fixture

- Prepares the environment **before a test runs**
- Cleans up **after** the test
- Provides **consistent input/data/state**

Mock

- Simulates behaviour of a dependency
- For example: API call

Example of a Fixture (pytest)

Function to be tested:

```python
import matplotlib.pyplot as plt

def plot_data(df):
    plt.plot(df["x"], df["y"])
    plt.xlabel("x")
    plt.ylabel("y")
    return plt
```

Test with fixture (providing df)

```python
# test_my_module.py
import pytest
import pandas as pd
from my_module import plot_data

@pytest.fixture
def example_dataframe():
    # This fixture creates reusable test data
    return pd.DataFrame({
        "x": [1, 2, 3],
        "y": [2, 4, 6]
    })

def test_plot_data_runs(example_dataframe):
    plt = plot_data(example_dataframe)
    assert plt.gca().has_data()
```

Pain points of software testing (in academia)

- Scientific software is often meant to establish new knowledge - known truth to test against
- Long execution times or pipelines
- Lack of knowledge of researchers about how to efficiently construct tests and prepare their code for tests
- Two code bases to maintain
- Incident model in academia: little focus on code, let alone additional code (tests)

Kanewala, U. & Bieman, J. M. Testing scientific software: A systematic literature review. *Inf. Softw. Technol.* **56**, 1219–1232 (2014).

Break

Activity 1 (40’)

Creating tests

Navigate to day2.1:

- Do Activity
  - Testing exercise
    (you may need to adjust paths to files)
  - feel free to ask Blablador or ChatGPT for help with programming ;)

Break

CI/CD

What do we need CI/CD for?

CI/CD

Continuos Integration /
Continuos Deployment

Ask yourself:

- When should your (unit-)tests be run?
- How do test your code runs on other computers (operating system, versions), too?

CI/CD

CI: continuous integration

- Runs tests, checks whether the code works etc

CD: continuous deployment

- Once all checks have passed, the code gets deployed, website gets built etc.

CI/CD: pipeline for robust output

GitHub Actions in Action Continuous integration and delivery for DevOps Michael Kaufmann, Rob Bos, and Marcel de Vries. Manning Publications 2023

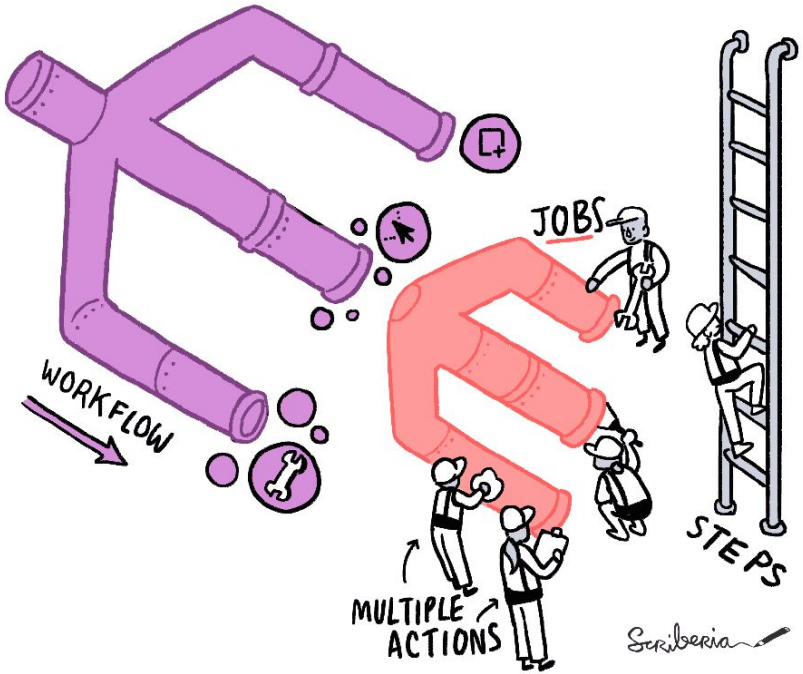
CI/CD

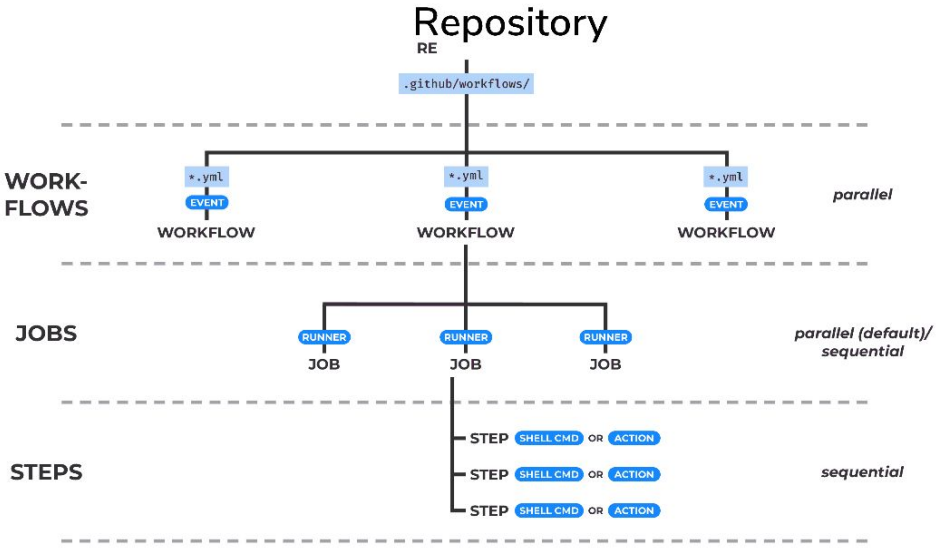
Scriberia

Github actions

- **GitHub Actions** are **automated workflows** that run when something happens in your GitHub repo — like pushing code, opening a pull request, or publishing a release.
- Often used as part of CI/CD pipeline
- Field: DevOps
- Examples:
  - Run tests every time code is pushed to main
  - Run quarto on push and render markdown to html on push
  - Open an issue every month
  - Close and issue with merging of a PR

Github Actions visualized





On the left: *The Turing Way* project illustration by Scriberia. Used under a CC-BY 4.0 licence. DOI: The Turing Way Community & Scriberia (2024). On the right: Overview diagram of the most important concepts of GitHub Actions, adapted from morioh.com.

Example

```yaml
name: Example GitHub Action

on:
  push:
    branches:
      - main
  workflow_dispatch:  # allows manual runs

jobs:
  hello-world:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Print a greeting
        run: echo "👋 Hello from GitHub Actions!"

      - name: Show basic environment info
        run: |
          echo "Current directory: $(pwd)"
          echo "Files in repo:"
          ls -la
          echo "GitHub actor: $GITHUB_ACTOR"

      - name: Run a simple script
        run: |
          for i in 1 2 3; do
            echo "This is step number $i"
            sleep 1
          done
          echo "✅ Workflow finished successfully!"
```

Example

https://github.com/fritjoflammers/github-actions-example

Activity 2:

Investigate a CI/CD pipeline

Go to https://github.com/fritjoflammers/nightingale

Make 3 groups: Each group investigates one of the three GH action workflow in the repository and present what they have found out

Activity 2b:

Creating a Github action workflow

Do Activity 2:

Introduction to Github actions.

WRAP UP